Day 160 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 23, Day 7 reference

Watch: Neural Collaborative Filtering (NCF) for recommendation systems explained (Data Science in your pocket) | Neural Collaborative Filtering (NCF) Explanation & Implementation in Pytorch (Steeve Huang)

`day160_neural_collaborative_filtering.py`, segment by segment, in the order `main()` uses things. The build day has no exercise file: the network parts (ncf_init, ncf_forward, ncf_loss_grad, ncf_train, ncf_scores, neumf_from) are new, and Days 154–159's parts are imported from their exercise files. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: F = every weight, in a dictionary · P, Q = GMF's user and film embeddings · U, V = the MLP's own embeddings · W0..W2, b0..b2 = the MLP's layers · h, c = output weights and bias · M = watched (True / False), users × films · lam = penalty

Segment 0 — Setup

Imports: the week's parts, kept quiet

CODE

```
HERE = os.path.dirname(os.path.abspath(__file__))
sys.path.insert(0, HERE)
with contextlib.redirect_stdout(io.StringIO()):
    import day154_exercise as d154
    import day155_exercise as d155
    import day156_exercise as d156
    import day157_exercise as d157
    import day158_exercise as d158
    import day159_exercise as d159
IALS = dict(k=50, lam=30, alpha=3)
STAR_ALPHA = 0.6
SVD_K = 20
NEG, LR, BATCH, EPOCHS = 4, 0.001, 256, 20
GMF_DS = (32, 64, 128)
GMF_LAMS = (0.003, 0.01, 0.03)
MLP_DMS = (32, 64, 128)
MLP_LAMS = (0.03, 0.1, 1.0)
```

- `with contextlib.redirect_stdout(io.StringIO()):` — everything printed inside the block goes into a string nobody reads: each exercise prints its hand checks when imported.
- `import day159_exercise as d159` — Day 159's measures, bootstrap, leave-last-out and sampled evaluation.
- `NEG, LR, BATCH, EPOCHS = 4, 0.001, 256, 20` — several names set in one line, from a tuple of values.

Segment 1 — The network

ncf_init: every weight in one dictionary

CODE

```
def ncf_init(nu, ni, d, dm, seed=0):
    rng = np.random.RandomState(seed); F = {}
    F["P"], F["Q"] = 0.01 * rng.randn(nu, d), 0.01 * rng.randn(ni, d)
    F["U"], F["V"] = 0.01 * rng.randn(nu, dm), 0.01 * rng.randn(ni, dm)
    sizes = (2 * dm, 2 * dm, dm, dm // 2)
    for l in range(3):
        F[f"W{l}"] = rng.randn(sizes[l], sizes[l + 1]) * np.sqrt(2.0 / sizes[l])
        F[f"b{l}"] = np.zeros(sizes[l + 1])
    F["h"] = 0.01 * rng.randn(d + dm // 2); F["c"] = np.zeros(1)
    return F
```

- `F["P"], F["Q"] = 0.01 * rng.randn(nu, d), ...` — small random embeddings, one row per user and per film.
- `sizes = (2 * dm, 2 * dm, dm, dm // 2)` — the MLP's widths: the joined embeddings, then layers 2d, d, d/2 (// keeps an integer).
- `F[f"W{l}"]` — an f-string key, so the layers are W0, W1, W2 in the same dictionary.
- `np.sqrt(2.0 / sizes[l])` — He initialisation: keeps the spread of a ReLU layer's output steady.
- `F["h"] = 0.01 * rng.randn(d + dm // 2)` — one output weight for each GMF number and each last-layer MLP number.

ncf_forward: GMF, MLP or both

MATH

$$GMF:\ x = p_u * q_i; \quad MLP:\ a_0 = [u_u, v_i],\ a_{l+1} = max(0, a_l W_l + b_l); \quad z = h \cdot [x, a_3] + c$$

CODE

```
def ncf_forward(F, u, i, mode):
    d = F["P"].shape[1]; C = {}
    g = F["P"][u] * F["Q"][i] if mode != "mlp" else np.zeros((len(u), d))
    if mode != "gmf":
        a = np.concatenate([F["U"][u], F["V"][i]], 1); C["acts"] = [a]
        for l in range(3):
            a = np.maximum(a @ F[f"W{l}"] + F[f"b{l}"], 0.0); C["acts"].append(a)
    else:
        a = np.zeros((len(u), F["h"].shape[0] - d))
    C["x"] = np.concatenate([g, a], 1)
    return C["x"] @ F["h"] + F["c"][0], C
```

- `F["P"][u] * F["Q"][i]` — fancy indexing picks one row per pair; * multiplies them number by number (not a dot product).
- `... if mode != "mlp" else np.zeros(...)` — a conditional expression: the unused tower contributes zeros.
- `np.concatenate([F["U"][u], F["V"][i]], 1)` — user and film embeddings side by side, along the columns.
- `np.maximum(a @ W + b, 0.0)` — a layer and its ReLU in one line.
- `C["acts"].append(a)` — every layer's output is kept: backprop needs them.
- `C["x"] @ F["h"] + F["c"][0]` — the logit: a weighted sum of both towers, plus a bias.

ncf_loss_grad: cross-entropy and hand-written backprop

MATH

$$loss = -mean[y \log p + (1-y) \log(1-p)] + (lam / 2B) \Sigma_{rows\ in\ batch} |row|^2; \quad \partial loss/\partial z = (p - y) / B$$

CODE

```
def ncf_loss_grad(F, u, i, y, mode, lam):
    z, C = ncf_forward(F, u, i, mode); B = len(y)
    p = 1.0 / (1.0 + np.exp(-z))
    loss = -np.mean(y * np.log(p + 1e-12) + (1 - y) * np.log(1 - p + 1e-12))
    emb = (("P", u), ("Q", i)) if mode == "gmf" else (("U", u), ("V", i)) if mode == "mlp" else
        (("P", u), ("Q", i), ("U", u), ("V", i))
    loss += 0.5 * lam * sum((F[k][idx] ** 2).sum() for k, idx in emb) / B
    G = {k: np.zeros_like(v) for k, v in F.items()}; dz = (p - y) / B
    G["h"] = C["x"].T @ dz; G["c"][0] = dz.sum(); dx = np.outer(dz, F["h"])
    d = F["P"].shape[1]
    if mode != "mlp":
        np.add.at(G["P"], u, dx[:, :d] * F["Q"][i]); np.add.at(G["Q"], i, dx[:, :d] * F["P"][u])
    if mode != "gmf":
        da = dx[:, d:]; acts = C["acts"]
        for l in (2, 1, 0):
            dl = da * (acts[l + 1] > 0)
            G[f"W{l}"] = acts[l].T @ dl; G[f"b{l}"] = dl.sum(0); da = dl @ F[f"W{l}"].T
        dm = F["U"].shape[1]
        np.add.at(G["U"], u, da[:, :dm]); np.add.at(G["V"], i, da[:, dm:])
    for k, idx in emb:
        np.add.at(G[k], idx, lam * F[k][idx] / B)
    return loss, G
```

- `1.0 / (1.0 + np.exp(-z))` — the sigmoid: a probability between 0 and 1.
- `np.log(p + 1e-12)` — a tiny number keeps log away from log(0).
- `emb = ... if ... else ... if ... else ...` — a chained conditional expression: which embedding tables this mode uses.
- `dz = (p - y) / B` — the gradient of mean cross-entropy with respect to each logit.
- `np.outer(dz, F["h"])` — the gradient arriving at x: every pair's dz times every output weight.
- `np.add.at(G["P"], u, ...)` — adds each row's gradient into its user; a user twice in the batch gets both. G["P"][u] += ... would keep only one.
- `for l in (2, 1, 0):` — backwards through the layers, last first.
- `da * (acts[l + 1] > 0)` — the ReLU passes gradient only where its output was positive.
- `np.add.at(G[k], idx, lam * F[k][idx] / B)` — the penalty's gradient, only on the rows the batch used.

ncf_train: fresh negatives each epoch, Adam on every weight

MATH

$$m \leftarrow 0.9m + 0.1g; \quad v \leftarrow 0.999v + 0.001g^2; \quad w \leftarrow w - lr \cdot (m/(1-0.9^t)) \,/\, (\sqrt{(v/(1-0.999^t))} + 10^{-8})$$

CODE

```
def ncf_train(F, M, mode, lam, epochs, lr=LR, seed=0, track=None, dense_l2=None):
    rng = np.random.RandomState(seed); pu, pi = np.nonzero(M); ni = M.shape[1]
    m = {k: np.zeros_like(v) for k, v in F.items()}; v = {k: np.zeros_like(x) for k, x in
        F.items()}; t = 0
    hist = []
    for _ in range(epochs):
        nu_ = np.repeat(pu, NEG); ni_ = rng.randint(0, ni, len(nu_)); bad = M[nu_, ni_]
        while bad.any():
            ni_[bad] = rng.randint(0, ni, bad.sum()); bad = M[nu_, ni_]
        uu, ii = np.r_[pu, nu_], np.r_[pi, ni_]; yy = np.r_[np.ones(len(pu)), np.zeros(len(nu_))]
        o = rng.permutation(len(uu)); tot = 0.0
        for s in range(0, len(o), BATCH):
            b = o[s:s + BATCH]; loss, G = ncf_loss_grad(F, uu[b], ii[b], yy[b], mode, 0.0 if
                dense_l2 else lam)
            tot += loss * len(b); t += 1
            for k in F:
                g = G[k] + (lam * F[k] if dense_l2 == "all" or (dense_l2 and k in ("P", "Q", "U",
                    "V")) else 0.0)
                m[k] = 0.9 * m[k] + 0.1 * g; v[k] = 0.999 * v[k] + 0.001 * g * g
                F[k] -= lr * (m[k] / (1 - 0.9 ** t)) / (np.sqrt(v[k] / (1 - 0.999 ** t)) + 1e-8)
        hist.append((tot / len(o),) + (track(F) if track else ()))
    return F, hist
```

- `pu, pi = np.nonzero(M)` — the row and column of every True: the watched pairs.
- `np.repeat(pu, NEG)` — each watched pair's user repeated 4 times, for 4 negatives.
- `while bad.any():` — redraw only the sampled films the user did watch, until none are left.
- `np.r_[pu, nu_]` — joins two arrays end to end.
- `rng.permutation(len(uu))` — a shuffled order of all pairs, then cut into batches of 256.
- `dense_l2 == "all" or (dense_l2 and k in (...))` — the first draft's decay, kept so section 2 can show what it did.
- `m[k] / (1 - 0.9 ** t)` — Adam's bias correction: early averages are scaled up.
- `(track(F) if track else ())` — the tracker's measures, or an empty tuple; added to the loss to make one record.

ncf_scores and neumf_from

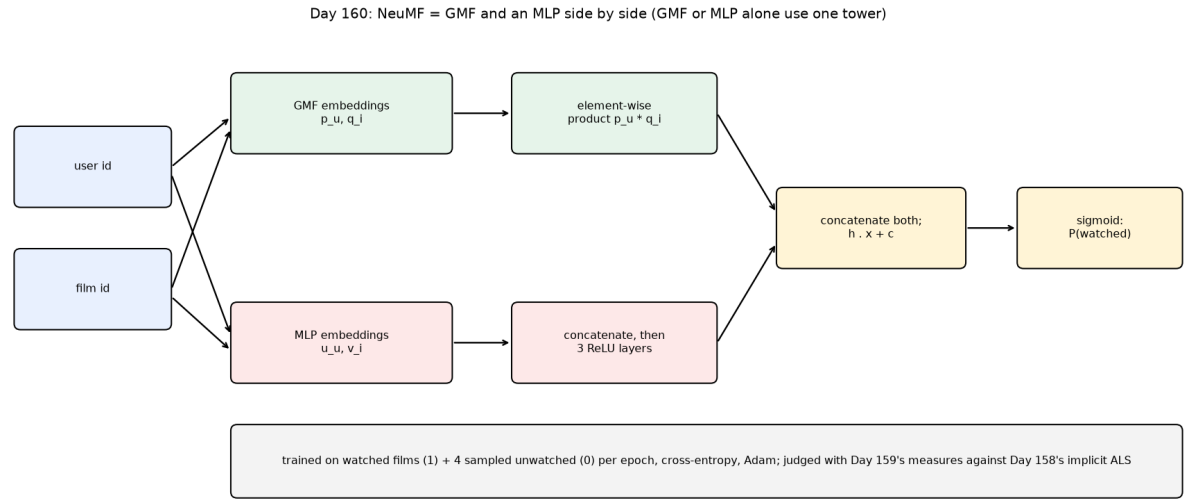
CODE

```
def ncf_scores(F, mode, nu, ni, chunk=64):
    S = np.zeros((nu, ni))
    for s in range(0, nu, chunk):
        n = min(s + chunk, nu) - s
        S[s:s + n] = ncf_forward(F, np.repeat(np.arange(s, s + n), ni), np.tile(np.arange(ni), n),
            mode)[0].reshape(n, ni)
    return S
def neumf_from(Fg, Fm):
    F = {k: x.copy() for k, x in Fm.items()}
    F["P"], F["Q"] = Fg["P"].copy(), Fg["Q"].copy()
    dg, dmm = Fg["P"].shape[1], Fm["P"].shape[1]
    F["h"] = np.r_[0.5 * Fg["h"][:dg], 0.5 * Fm["h"][dmm:]]
    F["c"] = 0.5 * (Fg["c"] + Fm["c"])
    return F
```

- `np.repeat(np.arange(s, s + n), ni)` — each user of the chunk repeated once per film ...
- `np.tile(np.arange(ni), n)` — ... against the list of all films repeated once per user: every pair.

- `... .reshape(n, ni)` — the long list of logits back into a users × films block.
- `{k: x.copy() for k, x in Fm.items()}` — a dictionary comprehension: copies, so fine-tuning cannot change the trained MLP.
- `np.r_[0.5 * Fg["h"][:dg], 0.5 * Fm["h"][dmm:]]` — half of each model's output weights, as in the paper's pre-training.

SYSTEM DIAGRAM



NeuMF: GMF and an MLP side by side, joined at the output.

Segment 2 — The experiments

Helpers: the data, the week's models, a tracker

CODE

```
def setup():
    u, i, r, t = d154.load_movielens(); _, genres, _ = d154.load_items()
    nu, ni = u.max() + 1, i.max() + 1
    tr, te = d154.split_ratings(len(r), 0.2, seed=0)
    fi, va = d154.split_ratings(len(tr), 0.1, seed=1); fi, va = tr[fi], tr[va]
    X = d156.film_features(genres, d156.load_years())[:, :19]
    return u, i, r, t, nu, ni, tr, te, fi, va, X
def watched(u, i, idx, nu, ni):
    M = np.zeros((nu, ni), bool); M[u[idx], i[idx]] = True
    return M
def gains(u, i, r, idx, nu, ni, graded=False):
    G = np.zeros((nu, ni)); G[u[idx], i[idx]] = 2.0 ** r[idx] - 1 if graded else 1.0
    return G
def week_models(u, i, r, idx, nu, ni):
    M = watched(u, i, idx, nu, ni); B = M.astype(float)
    R, _ = d154.to_matrix(u[idx], i[idx], r[idx], nu, ni)
    X, Y, _ = d158.implicit_als(np.where(M, 1.0 + IALS["alpha"], 0.0), IALS["k"], IALS["lam"])
    Xs, Ys, _ = d158.implicit_als(np.where(M, 1.0 + STAR_ALPHA * R, 0.0), IALS["k"], IALS["lam"])
    return M, {"popularity": np.broadcast_to(B.sum(0), M.shape),
               "co-occurrence": B @ d158.cooccurrence(B),
               "PureSVD": d155.low_rank(B, SVD_K),
               "implicit ALS": X @ Y.T,
               "implicit ALS, stars": Xs @ Ys.T}
def zrows(S):
    return (S - S.mean(1, keepdims=True)) / (S.std(1, keepdims=True) + 1e-12)
def val_precision(Mf, Hv, mode):
    return lambda F: (d159.per_user_metrics(ncf_scores(F, mode, *Mf.shape), Mf,
        Hv)[0]["precision"].mean(),)
```

- `np.broadcast_to(B.sum(0), M.shape)` — one row of popularity shown as a full table without copying it.
- `B @ d158.cooccurrence(B)` — Day 158's co-occurrence: films similar to the ones watched.
- `(S - S.mean(1, keepdims=True)) / ...` — each user's scores standardised, so two models can be blended on one scale.
- `return lambda F: (...,)` — a function made on the spot: called after every epoch, it returns a one-number tuple.

demo_tune: the grids, and NeuMF three ways

CODE

```
for how, F, lr in (("from scratch", ncf_init(nu, ni, dg, dmm), LR),
                   ("pre-trained", neumf_from(Fg, Fm), LR / 10),
                   ("pre-trained", neumf_from(Fg, Fm), LR / 100)):
    h0 = [(np.nan,) + val_precision(Mf, Hv, "neumf")(F)] if how == "pre-trained" else []
    F, h = ncf_train(F, Mf, "neumf", lg, EPOCHS, lr=lr, track=val_precision(Mf, Hv, "neumf"))
    h = h0 + h; v = [x[1] for x in h]; first = 0 if h0 else 1
    key = f"NeuMF, {how}, lr {lr:g}"; out[key] = (max(v), int(np.argmax(v)) + first, lr);
        curves[key] = h
    print(f"{key}: best validation {max(v):.4f} at epoch {out[key][1]}; by epoch from {first}: "
          f"{', '.join(f'{x:.3f}' for x in v)}")
```

- `for how, F, lr in ((...), (...), (...)):` — a loop over tuples, unpacked into three names each time.
- `[(np.nan,) + val_precision(...)(F)]` — epoch 0's record: no training loss yet (nan), then the validation score.
- `int(np.argmax(v)) + first` — the best epoch, counted from 0 when epoch 0 was scored, else from 1.

demo_final: the blend and the shuffled-user control

CODE

```
rng = np.random.RandomState(7); nu_ = np.repeat(u[va], NEG); ni_ = rng.randint(0, ni, len(nu_))
ok = ~Mf[nu_, ni_] & ~watched(u, i, va, nu, ni)[nu_, ni_]
cu, ci = np.r_[u[va], nu_[ok]], np.r_[i[va], ni_[ok]]; y = np.r_[np.ones(len(va)),
    np.zeros(ok.sum())]
w = d157.stack_fit(np.c_[zrows(Xv @ Yv.T)[cu, ci], zrows(nv)[cu, ci]], y)
print(f"Day 157's blend fitted on {len(y)} validation cells: intercept {w[0]:+.3f}, implicit ALS
    {w[1]:+.3f}, NeuMF {w[2]:+.3f}")
sc["blend: implicit ALS + NeuMF"] = d157.stack_predict(np.c_[zrows(sc["implicit ALS"]).ravel(),
    zrows(sc["NeuMF"]).ravel()], w).reshape(nu, ni)
perm = np.random.RandomState(0).permutation(nu); Fs = dict(Fn); Fs["P"], Fs["U"] = Fn["P"][perm],
    Fn["U"][perm]
sc["control: NeuMF, users shuffled"] = ncf_scores(Fs, "neumf", nu, ni)
```

- `ok = ~Mf[nu_, ni_] & ~watched(...)[nu_, ni_]` — ~ flips True and False; & keeps films unwatched in both tables.
- `d157.stack_fit(np.c_[a, b], y)` — Day 157's blend: least squares of the target on the two standardised scores.
- `Fs = dict(Fn); Fs["P"], Fs["U"] = Fn["P"][perm], ...` — a shallow copy with the user rows shuffled: each user gets someone else's embeddings.

demo_paper: one held-out film per user, and its noise

MATH

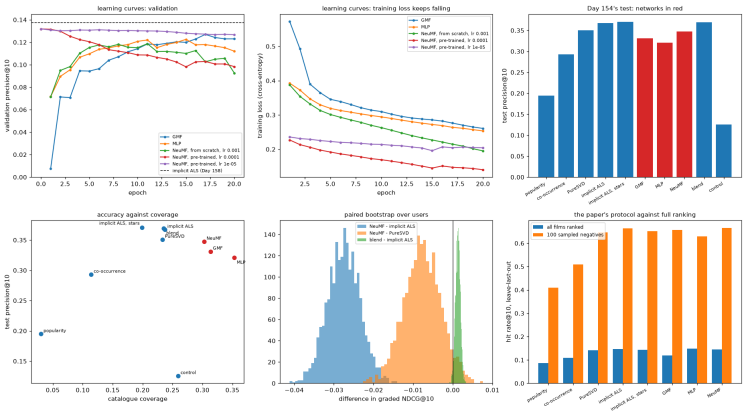
standard error of a hit rate = √(p(1 – p) / n), n = 943 users

CODE

```
item = np.zeros(nu, int); item[u[tel]] = i[tel]; G = np.zeros((nu, ni)); G[np.arange(nu), item] =
    1.0
print(f"{len(tel)} held-out films, models refitted ({time.time() - t0:.0f} s)")
print(f"{'':>20} {'all films ranked: HR@10':>24} {'NDCG@10':>8}   {'100 sampled: HR@10':>19}
    {'NDCG@10':>8}")
res = {}
for nm, S in sc.items():
    m = d159.per_user_metrics(S, M, G)[0]; hs, ns = d159.sampled_eval(S, M, item)
    res[nm] = (m["recall"].mean(), hs)
    print(f"{nm:>20} {m['recall'].mean():>24.4f} {m['ndcg'].mean():>8.4f}   {hs:>19.4f}
        {ns:>8.4f}")
for p_ in (0.15, 0.65):
    print(f"standard error of a hit rate near {p_:.2f} over {nu} users: {np.sqrt(p_ * (1 - p_) /
        nu):.4f}")
```

- `G[np.arange(nu), item] = 1.0` — row a, column item[a]: exactly one film to find per user.
- `d159.sampled_eval(S, M, item)` — Day 159's shortcut: the film's place among 100 sampled unwatched films.
- `np.sqrt(p_ * (1 - p_) / nu)` — the standard error of a share from nu yes/no results.

REAL GRAPH



Real graph from the script.

Quick Syntax Glossary — new or reinforced constructs

- `np.add.at(G, idx, x)` — add with repeated indices.
- `np.outer(a, b)` — every a times every b.
- `np.repeat / np.tile` — repeat items / the whole.
- `np.r_[a, b]` — join arrays end to end.
- `contextlib.redirect_stdout` — send prints elsewhere.
- `{k: f(x) for k, x in d.items()}` — dictionary comprehension.

End of Day 160 syntax reference. For the experiments and what they showed, see the Day 160 complete guide.